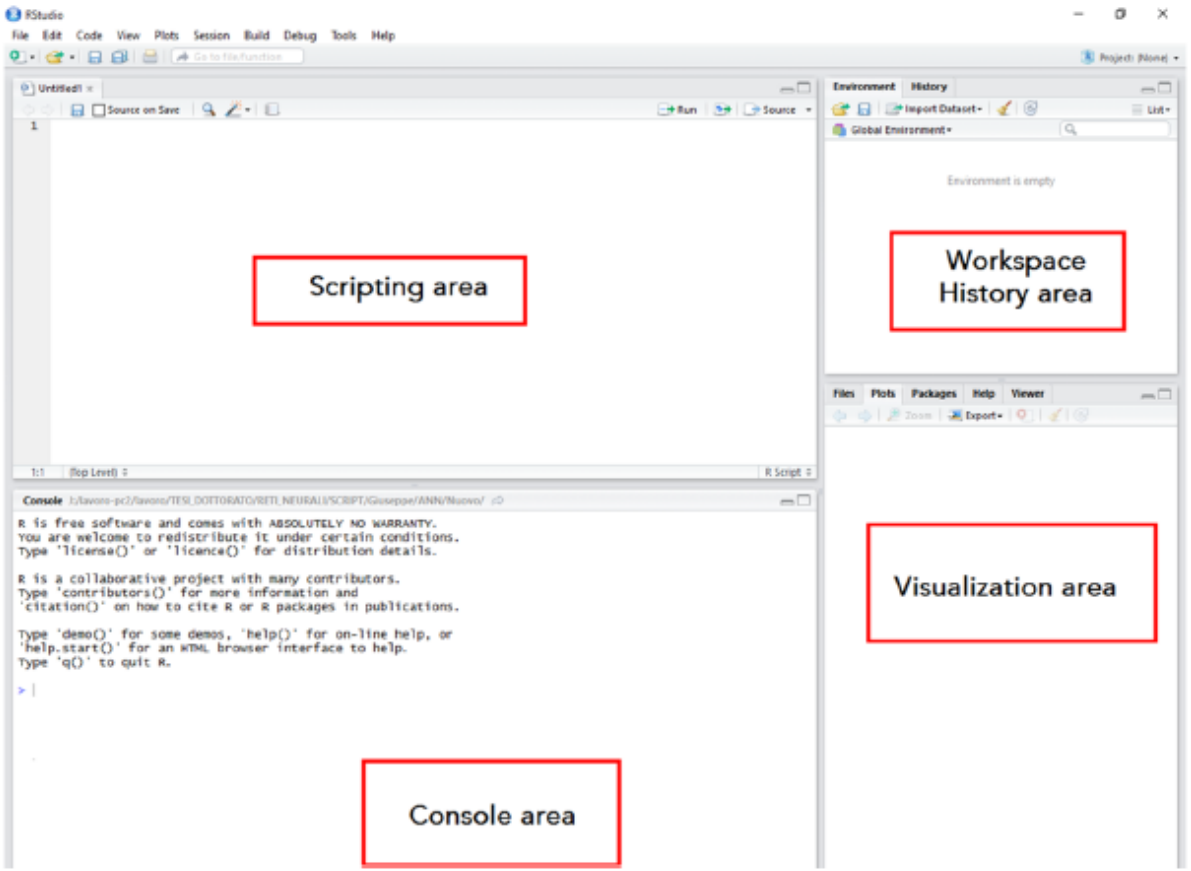
# Uygulamalı Regresyon_Ders1

# R studio


RStudio
File Edit Code View Plots Session Build Debug Tools Help
Scripting area
Environment History
Import Dataset
Global Environment
Environment is empty
Workspace
History area
Files Plots Packages Help Viewer
Export
Visualization area
Console
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
Console area

# R packages for regression

## The R stats package

R `stats` is a package that contains many useful functions for statistical calculations and random number generation. In the following table you will see some of the information on this package:

| | |
|---|---|
| **Package** | `stats` |
| **Date** | October 3, 2017 |
| **Version** | 3.5.0 |
| **Title** | The R stats package |
| **Author** | R core team and contributors worldwide |

- `lm`: This function is used to fit linear models. It can be used to carry out regression, single stratum analysis of variance, and analysis of co-variance.
- `summary.lm`: This function returns a summary for linear model fits.
- `coef`: With the help of this function, coefficients from objects returned by modeling functions can be extracted. Coefficients is an alias for it.
- `fitted`: Fitted values are extracted by this function from objects returned by modeling functions fitted. Values are an alias for it.
- `formula`: This function provides a way of extracting formulae which have been included in other objects.
- `predict`: This function predicts values based on linear model objects.
- `residuals`: This function extracts model residuals from objects returned by modeling functions.

- `confint`: This function computes confidence intervals for one or more parameters in a fitted model. Base has a method for objects inheriting from the `lm` class.
- `deviance`: This function returns the deviance of a fitted model object.
- `influence.measures`: This suite of functions can be used to compute some of the regression (leave-one-out deletion) diagnostics for linear and **generalized linear models (GLM)**.
- `lm.influence`: This function provides the basic quantities used when forming a wide variety of diagnostics for checking the quality of regression fits.
- `ls.diag`: This function computes basic statistics, including standard errors, t-values, and p-values for the regression coefficients.
- `glm`: This function is used to fit GLMs, specified by giving a symbolic description of the linear predictor and a description of the error distribution.
- `loess`: This function fits a polynomial surface determined by one or more numerical predictors, using local fitting.
- `loess.control`: This function sets control parameters for `loess` fits.
- `predict.loess`: This function extracts predictions from a `loess` fit, optionally with standard errors.
- `scatter.smooth`: This function plots and adds a smooth curve computed by `loess` to a scatter plot.

# The car package

This package includes many functions for: ANOVA analysis, matrix and vector transformations, printing readable tables of coefficients from several regression models, creating residual plots, tests for the autocorrelation of error terms, and many other general interest statistical and graphing functions.

In the following table you will see some of the information on this package:

| | |
|---|---|
| **Package** | `car` |
| **Date** | June 25, 2017 |
| **Version** | 2.1-5 |
| **Title** | Companion to Applied Regression |
| **Author** | John Fox, Sanford Weisberg, and many others |

- `Anova`: This function returns ANOVA tables for linear and GLMs
- `linear.hypothesis`: This function is used for testing a linear hypothesis and methods for linear models, GLMs, multivariate linear models, and linear and generalized linear mixed-effects models
- `cookd`: This function returns Cook's distances for linear and GLMs
- `outlier.test`: This function reports the Bonferroni p-values for studentized residuals in linear and GLMs, based on a t-test for linear models and a normal-distribution test for GLMs
- `durbin.watson`: This function computes residual autocorrelations and generalized Durbin-Watson statistics and their bootstrapped p-values
- `levene.test`: This function computes Levene's test for the homogeneity of variance across groups
- `ncv.test`: This function computes a score test of the hypothesis of constant error variance against the alternative that the error variance changes with the level of the response (fitted values), or with a linear combination of predictors

# The MASS package

This package includes many useful functions and data examples, including functions for estimating linear models through **generalized least squares** (**GLS**), fitting negative binomial linear models, the robust fitting of linear models, and Kruskal's non-metric multidimensional scaling.

In the following table you will see some of the information on this package:

| | |
|---|---|
| **Package** | `MASS` |
| **Date** | October 2, 2017 |
| **Version** | 7.3-47 |
| **Title** | Support Functions and Datasets for Venables and Ripley's MASS |
| **Author** | Brian Ripley, Bill Venables, and many others |

- `lm.gls`: This function fits linear models by GLS
- `lm.ridge`: This function fist a linear model by Ridge regression
- `glm.nb`: This function contains a modification of the system function
- `glm()`: It includes an estimation of the additional parameter, theta, to give a negative binomial GLM
- `polr`: A logistic or probit regression model to an ordered factor response is fitted by this function
- `lqs`: This function fits a regression to the good points in the dataset, thereby achieving a regression estimator with a high breakdown point
- `rlm`: This function fits a linear model by robust regression using an M-estimator
- `glmmPQL`: This function fits a GLMM model with multivariate normal random effects, using **penalized quasi-likelihood** (**PQL**)
- `boxcox`: This function computes and optionally plots profile log-likelihoods for the parameter of the **Box-Cox power transformation** for linear models

# The caret package

This package contains many functions to streamline the model training process for complex regression and classification problems. The package utilizes a number of R packages.

In the following table you will see listed some of the information on this package:

| | |
|---|---|
| **Package** | `caret` |
| **Date** | September 7, 2017 |
| **Version** | 6.0-77 |
| **Title** | Classification and Regression Training |
| **Author** | Max Kuhn and many others |

- `train`: Predictive models over different tuning parameters are fitted by this function. It fits each model, sets up a grid of tuning parameters for a number of classification and regression routines, and calculates a resampling-based performance measure.
- `trainControl`: This function permits the estimation of parameter coefficients with the help of resampling methods like cross-validation.
- `varImp`: This function calculates variable importance for the objects produced by train and method-specific methods.
- `defaultSummary`: This function calculates performance across resamples. Given two numeric vectors of data, the mean squared error and R-squared error are calculated. For two factors, the overall agreement rate and Kappa are determined.
- `knnreg`: This function performs **K-Nearest Neighbor** (**KNN**) regression that can return the average value for the neighbors.
- `plotObsVsPred`: This function plots observed versus predicted results in regression and classification models.
- `predict.knnreg`: This function extracts predictions from the KNN regression model.

# The glmnet package

This package contains many extremely efficient procedures in order to fit the entire Lasso or ElasticNet regularization path for linear regression, logistic and multinomial regression models, Poisson regression, and the Cox model. Multiple response Gaussian and grouped multinomial regression are the two recent additions.

In the following table you will see listed some of the information on this package:

| | |
|---|---|
| **Package** | `glmnet` |
| **Date** | September 21, 2017 |
| **Version** | 2.0-13 |
| **Title** | Lasso and Elastic-Net Regularized Generalized Linear Models |
| **Author** | Jerome Friedman, Trevor Hastie, Noah Simon, Junyang Qian, and Rob Tibshirani |

- `glmnet`: A GLM is fit by this function via penalized maximum likelihood. The regularization path is computed for the Lasso or ElasticNet penalty at a grid of values for the regularization parameter lambda. This function can also deal with all shapes of data, including very large sparse data matrices. Finally, it fits linear, logistic and multinomial, Poisson, and Cox regression models.
- `glmnet.control`: This function views and/or changes the factory default parameters in `glmnet`.
- `predict.glmnet`: This function predicts fitted values, logits, coefficients, and more from a fitted `glmnet` object.
- `print.glmnet`: This function prints a summary of the `glmnet` path at each step along the path.
- `plot.glmnet`: This function produces a coefficient profile plot of the coefficient paths for a fitted `glmnet` object.
- `deviance.glmnet`: This function computes the deviance sequence from the `glmnet` object.

As we have mentioned, this package fits Lasso and ElasticNet model paths for regression, logistic, and multinomial regression using coordinate descent. The algorithm is extremely fast, and exploits sparsity in the input matrix where it exists. A variety of predictions can be made from the fitted models.

# The sgd package

This package contains a fast and flexible set of tools for large scale estimation. It features many stochastic gradient methods, built-in models, visualization tools, automated hyperparameter tuning, model checking, interval estimation, and convergence diagnostics.

In the following table you will see listed some of the information on this package:

| | |
|---|---|
| **Package** | `sgd` |
| **Date** | January 5, 2016 |
| **Version** | 1.1 |
| **Title** | Stochastic Gradient Descent for Scalable Estimation |
| **Author** | Dustin Tran, Panos Toulis, Tian Lian, Ye Kuang, and Edoardo Airoldi |

- `sgd`: This function runs **Stochastic Gradient Descent** (**SGD**) in order to optimize the induced loss function given a model and data
- `print.sgd`: This function prints objects of the `sgd` class
- `predict.sgd`: This function forms predictions using the estimated model parameters from SGD
- `plot.sgd`: This function `plots` objects of the `sgd` class

# The BLR package

This package performs a special case of linear regression named Bayesian linear regression. In Bayesian linear regression, the statistical analysis is undertaken within the context of a Bayesian inference.

In the following table you will see listed some of the information on this package:

| | |
|---|---|
| **Package** | `BLR` |
| **Date** | December 3, 2014 |
| **Version** | 1.4 |
| **Title** | Bayesian Linear Regression |
| **Author** | Gustavo de los Campos, Paulino Perez Rodriguez |

- `BLR`: This function was designed to fit parametric regression models using different types of shrinkage methods.
- `sets`: This is a vector (*599x1*) that assigns observations to ten disjointed sets; the assignment was generated at random. This is used later to conduct a 10-fold CV.

# The Lars package

This package contains efficient procedures for fitting an entire Lasso sequence with the cost of a single least squares fit. Least angle regression and infinitesimal forward stagewise regression are related to the Lasso.

In the following table you will see listed some of the information on this package:

| | |
|---|---|
| **Package** | `Lars` |
| **Date** | April 23, 2013 |
| **Version** | 1.2 |
| **Title** | Least Angle Regression, Lasso and Forward Stagewise |
| **Author** | Trevor Hastie and Brad Efron |

- `lars`: This function fits least angle regression and Lasso and infinitesimal forward stagewise regression models.
- `summary.lars`: This function produces an ANOVA-type summary for a `lars` object.
- `plot.lars`: This function produce a plot of a `lars` fit. The default is a complete coefficient path.
- `predict.lars`: This function make predictions or extracts coefficients from a fitted `lars` model.

## C.1 Reading the data in

The first step is to read the data in. You can use the `read.table()` or `scan()` function to read data in from outside R. You can also use the `data()` function to access data already available within R.

```
> data(stackloss)
> stackloss
   Air.Flow Water.Temp Acid.Conc. stack.loss
1        80         27         89         42
2        80         27         88         37
... stuff deleted ...
21       70         20         91         15
```

Type

```
> help(stackloss)
```

We can check the dimension of the data:

```
> dim(stackloss)
[1] 21  4
```

## C.2 Numerical Summaries

One easy way to get the basic numerical summaries is:

```
> summary(stackloss)
    Air.Flow       Water.Temp     Acid.Conc.      stack.loss
 Min.   :50.0   Min.   :17.0   Min.   :72.0   Min.   : 7.0
 1st Qu.:56.0   1st Qu.:18.0   1st Qu.:82.0   1st Qu.:11.0
 Median :58.0   Median :20.0   Median :87.0   Median :15.0
 Mean   :60.4   Mean   :21.1   Mean   :86.3   Mean   :17.5
 3rd Qu.:62.0   3rd Qu.:24.0   3rd Qu.:89.0   3rd Qu.:19.0
 Max.   :80.0   Max.   :27.0   Max.   :93.0   Max.   :42.0
```

We can compute these numbers seperately also:

```
> stackloss$Air.Flow
 [1] 80 80 75 62 62 62 62 62 58 58 58 58 58 58 50 50 50 50 50 56 70
> mean(stackloss$Ai)
[1] 60.429
> median(stackloss$Ai)
[1] 58
> range(stackloss$Ai)
[1] 50 80
> quantile(stackloss$Ai)
  0%  25%  50%  75% 100%
  50   56   58   62   80
```

We can get the variance and sd:

```
> var(stackloss$Ai)
[1] 84.057
> sqrt(var(stackloss$Ai))
[1] 9.1683
```

We can write a function to compute sd's:

```
> sd <- function(x) sqrt(var(x))
> sd(stackloss$Ai)
[1] 9.1683
```

We might also want the correlations:

```
> cor(stackloss)
            Air.Flow Water.Temp Acid.Conc. stack.loss
Air.Flow    1.00000    0.78185    0.50014    0.91966
Water.Temp  0.78185    1.00000    0.39094    0.87550
Acid.Conc.  0.50014    0.39094    1.00000    0.39983
stack.loss  0.91966    0.87550    0.39983    1.00000
```

We might also want the correlations:

```
> cor(stackloss)
            Air.Flow Water.Temp Acid.Conc. stack.loss
Air.Flow     1.00000    0.78185    0.50014    0.91966
Water.Temp   0.78185    1.00000    0.39094    0.87550
Acid.Conc.   0.50014    0.39094    1.00000    0.39983
stack.loss   0.91966    0.87550    0.39983    1.00000
```

Another numerical summary with a graphical element is the stem plot:

```
> stem(stackloss$Ai)

  The decimal point is 1 digit(s) to the right of the |

  5 | 000006888888
  6 | 22222
  7 | 05
  8 | 00
```

## C.3 Graphical Summaries

We can make histograms and boxplot and specify the labels if we like:

```
> hist(stackloss$Ai)
> hist(stackloss$Ai,main="Histogram of Air Flow",
  xlab="Flow of cooling air")
> boxplot(stackloss$Ai)
```

Scatterplots are also easily constructed:

```
> plot(stackloss$Ai,stackloss$W)
> plot(Water.Temp ~ Air.Flow,stackloss,xlab="Air Flow",
  ylab="Water Temperature")
```

We can make a scatterplot matrix:

```
> plot(stackloss)
```

We can put several plots in one display

```
> par(mfrow=c(2,2))
> boxplot(stackloss$Ai)
> boxplot(stackloss$Wa)
> boxplot(stackloss$Ac)
> boxplot(stackloss$s)
> par(mfrow=c(1,1))
```

## C.4 Selecting subsets of the data

Second row:

```
> stackloss[2,]
  Air.Flow Water.Temp Acid.Conc. stack.loss
2       80         27         88         37
```

Third column:

```
> stackloss[,3]
 [1] 89 88 90 87 87 87 93 93 87 80 89 88 82 93 89 86 72 79 80 82 91
```

The 2,3 element:

```
> stackloss[2,3]
[1] 88
```

c() is a function for making vectors, e.g.

```
> c(1,2,4)
[1] 1 2 4
```

Select the first, second and fourth rows:

```
> stackloss[c(1,2,4),]
  Air.Flow Water.Temp Acid.Conc. stack.loss
1       80         27         89         42
2       80         27         88         37
4       62         24         87         28
```

The : operator is good for making sequences e.g.

```
> 3:11
[1]  3  4  5  6  7  8  9 10 11
```

We can select the third through sixth rows:

```
> stackloss[3:6,]
  Air.Flow Water.Temp Acid.Conc. stack.loss
3       75         25         90         37
4       62         24         87         28
5       62         22         87         18
6       62         23         87         18
```

We can use "-" to indicate "everthing but", e.g all the data except the first two columns is:

```
> stackloss[,-c(1,2)]
   Acid.Conc. stack.loss
1          89         42
2          88         37
... stuff deleted ...
21         91         15
```

We may also want select the subsets on the basis of some criterion e.g. which cases have an air flow greater than 72.

```
> stackloss[stackloss$Ai > 72,]
  Air.Flow Water.Temp Acid.Conc. stack.loss
1       80         27         89         42
2       80         27         88         37
3       75         25         90         37
```

- Let’s look at an example. The National Institute of Diabetes and Digestive and Kidney Diseases conducted a study on 768 adult female Pima Indians living near Phoenix. The following variables were recorded:
- Number of times pregnant,
- Plasma glucose concentration a 2 hours in an oral glucose tolerance test,
- Diastolic blood pressure (mm Hg),
- Triceps skin fold thickness (mm),
- 2-Hour serum insulin (mu U/ml),
- Body mass index (weight in kg/(height in $m2$)),
- Diabetes pedigree function,
- Age (years) and
- a test whether the patient shows signs of diabetes (coded 0 if negative, 1 if positive).

```
> library(faraway)
> data(pima)
> pima
```

```
> summary(pima)
    pregnant         glucose       diastolic        triceps          insulin
 Min.   : 0.00   Min.   :  0   Min.   :  0.0   Min.   : 0.0   Min.   :  0.0
 1st Qu.: 1.00   1st Qu.: 99   1st Qu.: 62.0   1st Qu.: 0.0   1st Qu.:  0.0
 Median : 3.00   Median :117   Median : 72.0   Median :23.0   Median : 30.5
 Mean   : 3.85   Mean   :121   Mean   : 69.1   Mean   :20.5   Mean   : 79.8
 3rd Qu.: 6.00   3rd Qu.:140   3rd Qu.: 80.0   3rd Qu.:32.0   3rd Qu.:127.2
 Max.   :17.00   Max.   :199   Max.   :122.0   Max.   :99.0   Max.   :846.0
      bmi          diabetes          age             test
 Min.   : 0.0   Min.   :0.078   Min.   :21.0   Min.   :0.000
 1st Qu.:27.3   1st Qu.:0.244   1st Qu.:24.0   1st Qu.:0.000
 Median :32.0   Median :0.372   Median :29.0   Median :0.000
 Mean   :32.0   Mean   :0.472   Mean   :33.2   Mean   :0.349
 3rd Qu.:36.6   3rd Qu.:0.626   3rd Qu.:41.0   3rd Qu.:1.000
 Max.   :67.1   Max.   :2.420   Max.   :81.0   Max.   :1.000
```

The `summary()` command is a quick way to get the usual univariate summary information. At this stage, we are looking for anything unusual or unexpected perhaps indicating a data entry error. For this purpose, a close look at the minimum and maximum values of each variable is worthwhile. Starting with `pregnant`, we see a maximum value of 17. This is large but perhaps not impossible. However, we then see that the next 5 variables have minimum values of zero. No blood pressure is not good for the health — something must be wrong. Let's look at the sorted values:

```
> sort(pima$diastolic)
  [1]   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
 [19]   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  24
 [37]  30  30  38  40  44  44  44  44  46  46  48  48  48  48  48  50  50  50
...etc...
```

We see that the first 36 values are zero. The description that comes with the data says nothing about it but it seems likely that the zero has been used as a missing value code. For one reason or another, the researchers did not obtain the blood pressures of 36 patients. In a real investigation, one would likely be able to question the researchers about what really happened. Nevertheless, this does illustrate the kind of misunderstanding

We set all zero values of the five variables to NA which is the missing value code used by R .

```
> pima$diastolic[pima$diastolic == 0]  <- NA
> pima$glucose[pima$glucose == 0] <- NA
> pima$triceps[pima$triceps == 0]  <- NA
> pima$insulin[pima$insulin == 0] <- NA
> pima$bmi[pima$bmi == 0] <- NA
```

The variable test is not quantitative but categorical. Such variables are also called *factors*. However, because of the numerical coding, this variable has been treated as if it were quantitative. It's best to designate such variables as factors so that they are treated appropriately. Sometimes people forget this and compute stupid statistics such as "average zip code".

```
> pima$test <- factor(pima$test)
> summary(pima$test)
  0   1
500 268
```

We now see that 500 cases were negative and 268 positive. Even better is to use descriptive labels:

```
> levels(pima$test) <- c("negative","positive")
> summary(pima)
    pregnant         glucose       diastolic        triceps          insulin
 Min.   : 0.00   Min.   : 44   Min.   : 24.0   Min.   :  7.0   Min.   : 14.0
 1st Qu.: 1.00   1st Qu.: 99   1st Qu.: 64.0   1st Qu.: 22.0   1st Qu.: 76.2
 Median : 3.00   Median :117   Median : 72.0   Median : 29.0   Median :125.0
 Mean   : 3.85   Mean   :122   Mean   : 72.4   Mean   : 29.2   Mean   :155.5
 3rd Qu.: 6.00   3rd Qu.:141   3rd Qu.: 80.0   3rd Qu.: 36.0   3rd Qu.:190.0
 Max.   :17.00   Max.   :199   Max.   :122.0   Max.   : 99.0   Max.   :846.0
                 NA's   :  5   NA's   : 35.0   NA's   :227.0   NA's   :374.0
      bmi          diabetes          age             test
 Min.   :18.2   Min.   :0.078   Min.   :21.0   negative:500
 1st Qu.:27.5   1st Qu.:0.244   1st Qu.:24.0   positive:268
 Median :32.3   Median :0.372   Median :29.0
 Mean   :32.5   Mean   :0.472   Mean   :33.2
 3rd Qu.:36.6   3rd Qu.:0.626   3rd Qu.:41.0
 Max.   :67.1   Max.   :2.420   Max.   :81.0
 NA's   :11.0
```

Now that we've cleared up the missing values and coded the data appropriately we are ready to do some plots. Perhaps the most well-known univariate plot is the histogram:

```
hist(pima$diastolic)
```

We see a bell-shaped distribution for the diastolic blood pressures centered around 70. The construction of a histogram requires the specification of the number of bins and their spacing on the horizontal axis. Some choices can lead to histograms that obscure some features of the data. R attempts to specify the number and spacing of bins given the size and distribution of the data but this choice is not foolproof and misleading histograms are possible. For this reason, I prefer to use Kernel Density Estimates which are essentially a smoothed version of the histogram.

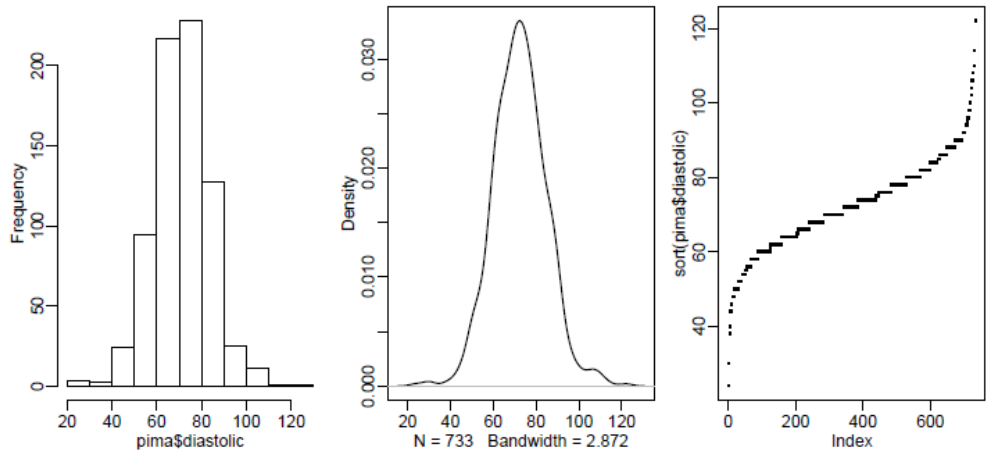



Figure 1.1: First panel shows histogram of the diastolic blood pressures, the second shows a kernel density estimate of the same while the the third shows an index plot of the sorted values

An alternative is to simply plot the sorted data against its index:
plot(sort(pima$diastolic),pch=".")

The advantage of this is we can see all the data points themselves. We can see the distribution and possible outliers. We can also see the discreteness in the measurement of blood pressure - values are rounded to the nearest even number and hence we the “steps” in the plot. The advantage of this is we can see all the data points themselves. We can see the distribution and possible outliers. We can also see the discreteness in the measurement of blood pressure - values are rounded to the nearest even number and hence we the “steps” in the plot. A couple of bivariate plots as seen in Figure:
> plot(diabetes ~ diastolic,pima)
> plot(diabetes ~ test,pima)
hist(pima$diastolic)
First, we see the standard scatterplot showing two quantitative variables. Second, we see a side-by-side boxplot suitable for showing a quantitative and a qualititative variable. Also useful is a scatterplot matrix, produced by
> pairs(pima)

## Regression Analysis

Regression analysis is used for explaining or modeling the relationship between a single variable *Y*, calledt he *response*, *output* or *dependent* variable, and one or more *predictor*, *input*, *independent* or *explanatory* variables, $X_1$ , $X_2$ …. $X_{p.}$ When p = 1 it is called simple regression but when p > 1 it is called multiple regression or sometimes multivariate regression. When there is more than one Y, then it is called multivariate (multiple) regression.

The response must be a continuous variable but the explanatory variables can be continuous, discrete or categorical although we leave the handling of categorical explanatory variables. Taking the example presented above, a regression of diastolic and bmi on diabetes would be a multiple regression involving only quantitative variables. A regression of diastolic and bmi on test would involve one predictor which is quantitative which we will consider *Analysis of Covariance*. A regression of diastolic on just test would involve just qualitative predictors, a topic called *Analysis of Variance* or *ANOVA* although this would just be a simple two sample situation. A regression of test (the response) on diastolic and bmi (the predictors) would involve a qualitative response, a *logistic regression* could be used.

Regression analyses have several possible objectives including:

- 1. Prediction of future observations.
- 2. Assessment of the effect of, or relationship between, explanatory variables on the response.
- 3. A general description of data structure.

Extensions exist to handle multivariate responses, binary responses (logistic regression analysis) and count responses (poisson regression).

## History

Regression-type problems were first considered in the 18th century concerning navigation using astronomy. Legendre developed the method of least squares in 1805. Francis Galton coined the term *regression to mediocrity* in 1875 in reference to the simple regression equation in the form

$$\frac{y-\bar{y}}{SD_y} = r\frac{(x-\bar{x})}{SD_x}.$$

Galton used this equation to explain the phenomenon that sons of tall fathers tend to be tall but not as tall as their fathers while sons of short fathers tend to be short but not as short as their fathers. This effect is called the *regression effect*.

$$y = rx$$

```
> data(stat500)
> stat500 <- data.frame(scale(stat500))
> plot(final ~ midterm,stat500)
> abline(0,1)
```

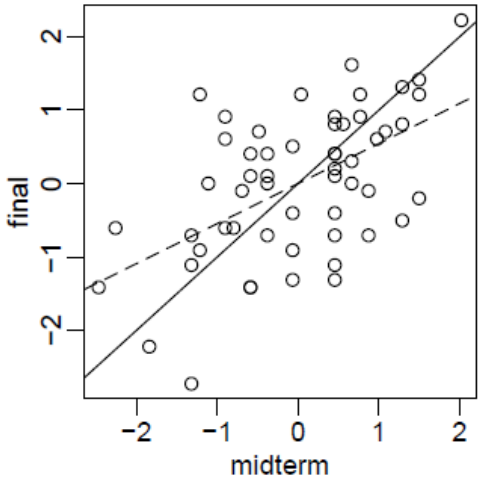



Figure 1.3: Final and midterm scores in standard units. Least squares fit is shown with a dotted line while $y = x$ is shown as a solid line

We have added the y=x x (solid) line to the plot. Now a student scoring, say one standard deviation above average on the midterm might reasonably expect to do equally well on the final. We compute the least squares regression fit and plot the regression line (more on the details later). We also compute the correlations.

```
> g <- lm(final ~ midterm,stat500)
> abline(g$coef,lty=5)
> cor(stat500)
         midterm    final       hw   total
midterm 1.00000 0.545228 0.272058 0.84446
final   0.54523 1.000000 0.087338 0.77886
hw      0.27206 0.087338 1.000000 0.56443
total   0.84446 0.778863 0.564429 1.00000
```

Estimation

## 2.1 Example

Let's start with an example. Suppose that $Y$ is the fuel consumption of a particular model of car in m.p.g. Suppose that the predictors are

1. $X_1$ — the weight of the car
2. $X_2$ — the horse power
3. $X_3$ — the no. of cylinders.

$X_3$ is discrete but that's OK. Using country of origin, say, as a predictor would not be possible within the current development (we will see how to do this later in the course). Typically the data will be available in the form of an array like this

$$\begin{array}{cccc} y_1 & x_{11} & x_{12} & x_{13} \\ y_2 & x_{21} & x_{22} & x_{23} \\ \ldots & & \ldots & \\ y_n & x_{n1} & x_{n2} & x_{n3} \end{array}$$

where $n$ is the number of observations or *cases* in the dataset.

## 2.2 Linear Model

One very general form for the model would be

$$Y = f(X_1, X_2, X_3) + \varepsilon$$

where $f$ is some unknown function and $\varepsilon$ is the error in this representation which is additive in this instance. Since we usually don't have enough data to try to estimate $f$ directly, we usually have to assume that it has some more restricted form, perhaps linear as in

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon$$

where $\beta_i$, $i = 0, 1, 2, 3$ are unknown *parameters*. $\beta_0$ is called the *intercept* term. Thus the problem is reduced to the estimation of four values rather than the complicated infinite dimensional $f$.

In a linear model the *parameters enter linearly* — the predictors do not have to be linear. For example

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 \log X_2 + \varepsilon$$

is linear but

$$Y = \beta_0 + \beta_1 X_1^{\beta_2} + \varepsilon$$

is not. Some relationships can be transformed to linearity — for example $y = \beta_0 x_1^{\beta} \varepsilon$ can be linearized by taking logs. Linear models seem rather restrictive but because the predictors can transformed and combined in any way, they are actually very flexible. Truly non-linear models are rarely absolutely necessary and most often arise from a theory about the relationships between the variables rather than an empirical investigation.

## 2.3 Matrix Representation

Given the actual data, we may write

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 x_{3i} + \varepsilon_i \quad i = 1, \ldots, n$$

but the use of subscripts becomes inconvenient and conceptually obscure. We will find it simpler both notationally and theoretically to use a matrix/vector representation. The regression equation is written as

$$y = X\beta + \varepsilon$$

where $y = (y_1 \ldots y_n)^T$, $\varepsilon = (\varepsilon_1 \ldots \varepsilon_n)^T$, $\beta = (\beta_0 \ldots \beta_3)^T$ and

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & x_{13} \\ 1 & x_{21} & x_{22} & x_{23} \\ \ldots & & \ldots & \\ 1 & x_{n1} & x_{n2} & x_{n3} \end{pmatrix}$$

The column of ones incorporates the intercept term. A couple of examples of using this notation are the simple no predictor, mean only model $y = \mu + \varepsilon$

$$\begin{pmatrix} y_1 \\ \ldots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 \\ \ldots \\ 1 \end{pmatrix} \mu + \begin{pmatrix} \varepsilon_1 \\ \ldots \\ \varepsilon_n \end{pmatrix}$$

We can assume that $E\varepsilon = 0$ since if this were not so, we could simply absorb the non-zero expectation for the error into the mean $\mu$ to get a zero expectation. For the two sample problem with a treatment group having the response $y_1, \ldots, y_m$ with mean $\mu_y$ and control group having response $z_1, \ldots, z_n$ with mean $\mu_z$ we have

$$\begin{pmatrix} y_1 \\ \ldots \\ y_m \\ z_1 \\ \ldots \\ z_n \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ \ldots & \\ 1 & 0 \\ 0 & 1 \\ . & . \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \mu_y \\ \mu_z \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \ldots \\ \ldots \\ \ldots \\ \varepsilon_{m+n} \end{pmatrix}$$

## 2.4 Estimating β

We have the regression equation $y = X\beta + \varepsilon$ - what estimate of β would best separate the systematic component $X\beta$ from the random component ε. Geometrically speaking, $y \in \mathbb{R}^n$ while $\beta \in \mathbb{R}^p$ where $p$ is the number of parameters (if we include the intercept then $p$ is the number of predictors plus one).

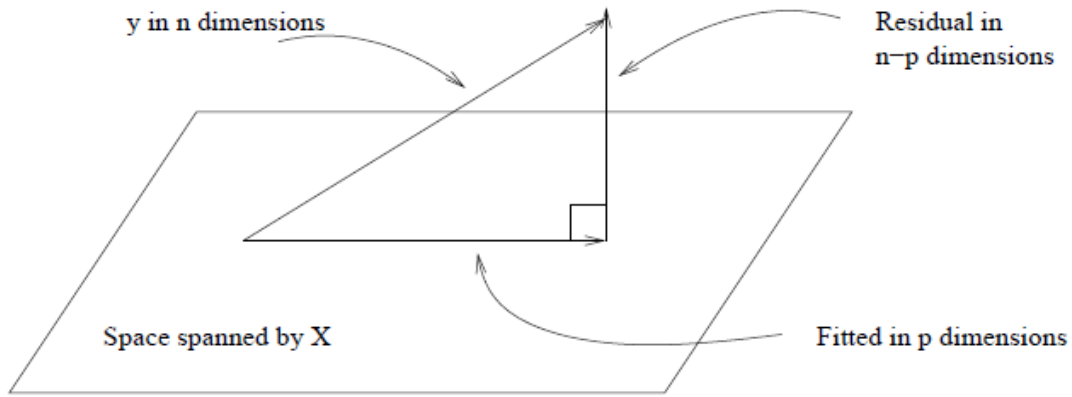



Figure 2.1: Geometric representation of the estimation β. The data vector Y is projected orthogonally onto the model space spanned by $X$. The fit is represented by projection $\hat{y} = X\hat{\beta}$ with the difference between the fit and the data represented by the residual vector $\hat{\varepsilon}$.

The problem is to find β such that $X\beta$ is close to $Y$. The best choice of $\hat{\beta}$ is apparent in the geometrical representation shown in Figure 2.4.

$\hat{\beta}$ is in some sense the best estimate of β within the model space. The response predicted by the model is $\hat{y} = X\hat{\beta}$ or $Hy$ where $H$ is an orthogonal projection matrix. The difference between the actual response and the predicted response is denoted by $\hat{\varepsilon}$ — the residuals.

The conceptual purpose of the model is to represent, as accurately as possible, something complex — $y$ which is $n$-dimensional — in terms of something much simpler — the model which is $p$-dimensional. Thus if our model is successful, the structure in the data should be captured in those $p$ dimensions, leaving just random variation in the residuals which lie in an $n - p$ dimensional space. We have

$$\begin{aligned} \text{Data} &= \text{Systematic Structure} + \text{Random Variation} \\ n \text{ dimensions} &= p \text{ dimensions} + (n-p) \text{ dimensions} \end{aligned}$$

## 2.5 Least squares estimation

The estimation of $\beta$ can be considered from a non-geometric point of view. We might define the best estimate of $\beta$ as that which minimizes the sum of the squared errors, $\varepsilon^T\varepsilon$. That is to say that the least squares estimate of $\beta$, called $\hat{\beta}$ minimizes

$$\sum \varepsilon_i^2 = \varepsilon^T\varepsilon = (y - X\beta)^T(y - X\beta)$$

Expanding this out, we get

$$y^Ty - 2\beta X^Ty + \beta^T X^T X\beta$$

Differentiating with respect to $\beta$ and setting to zero, we find that $\hat{\beta}$ satisfies

$$X^TX\hat{\beta} = X^Ty$$

These are called the normal equations. We can derive the same result using the geometric approach. Now provided $X^TX$ is invertible

$$\begin{aligned} \hat{\beta} &= (X^TX)^{-1}X^Ty \\ X\hat{\beta} &= X(X^TX)^{-1}X^Ty \\ &= Hy \end{aligned}$$

$H = X(X^TX)^{-1}X^T$ is called the "hat-matrix" and is the orthogonal projection of $y$ onto the space spanned by $X$. $H$ is useful for theoretical manipulations but you usually don't want to compute it explicitly as it is an $n \times n$ matrix.

- Predicted values: $\hat{y} = Hy = X\hat{\beta}$.
- Residuals: $\hat{\varepsilon} = y - X\hat{\beta} = y - \hat{y} = (I - H)y$
- Residual sum of squares: $\hat{\varepsilon}^T\hat{\varepsilon} = y^T(I - H)(I - H)y = y^T(I - H)y$

Later we will show that the least squares estimate is the best possible estimate of $\beta$ when the errors $\varepsilon$ are uncorrelated and have equal variance - i.e. var $\varepsilon = \sigma^2 I$.

## 2.6 Examples of calculating $\hat{\beta}$

1. When $y = \mu + \varepsilon$, $X = \mathbf{1}$ and $\beta = \mu$ so $X^T X = \mathbf{1}^T \mathbf{1} = n$ so

$$\hat{\beta} = (X^T X)^{-1} X^T y = \frac{1}{n} \mathbf{1}^T y = \bar{y}$$

2. Simple linear regression (one predictor)

$$y_i = \alpha + \beta x_i + \varepsilon_i$$

$$\begin{pmatrix} y_1 \\ \dots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ \dots & \\ 1 & x_n \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \dots \\ \varepsilon_n \end{pmatrix}$$

We can now apply the formula but a simpler approach is to rewrite the equation as

$$y_i = \overbrace{\alpha + \beta \bar{x}}^{\alpha'} + \beta(x_i - \bar{x}) + \varepsilon_i$$

so now

$$X = \begin{pmatrix} 1 & x_1 - \bar{x} \\ \dots & \\ 1 & x_n - \bar{x} \end{pmatrix} \qquad X^T X = \begin{pmatrix} n & 0 \\ 0 & \sum_{i=1}^n (x_i - \bar{x})^2 \end{pmatrix}$$

Now work through the rest of the calculation to reconstruct the familiar estimates, i.e.

$$\hat{\beta} = \frac{\sum (x_i - \bar{x}) y}{\sum (x_i - \bar{x})^2}$$

In higher dimensions, it is usually not possible to find such explicit formulae for the parameter estimates unless $X^T X$ happens to be a simple form.

## 2.7 Why is $\hat{\beta}$ a good estimate?

1. It results from an orthogonal projection onto the model space. It makes sense geometrically.
2. If the errors are independent and identically normally distributed, it is the maximum likelihood estimator. Loosely put, the maximum likelihood estimate is the value of $\beta$ that maximizes the probability of the data that was observed.
3. The Gauss-Markov theorem states that it is best linear unbiased estimate. (BLUE).

**Implications**

The Gauss-Markov theorem shows that the least squares estimate $\hat{\beta}$ is a good choice, but if the errors are correlated or have unequal variance, there will be better estimators. Even if the errors behave but are non-normal then non-linear or biased estimates may work better in some sense. So this theorem does not tell one to use least squares all the time, it just strongly suggests it unless there is some strong reason to do otherwise.

Situations where estimators other than ordinary least squares should be considered are

1. When the errors are correlated or have unequal variance, generalized least squares should be used.
2. When the error distribution is long-tailed, then robust estimates might be used. Robust estimates are typically not linear in $y$.
3. When the predictors are highly correlated (collinear), then biased estimators such as ridge regression might be preferable.

## 2.9 Mean and Variance of $\hat{\beta}$

Now $\hat{\beta} = (X^T X)^{-1} X^T y$ so

- Mean $E\hat{\beta} = (X^T X)^{-1} X^T X\beta = \beta$ (unbiased)
- var $\hat{\beta} = (X^T X)^{-1} X^T \sigma^2 I X (X^T X)^{-1} = (X^T X)^{-1} \sigma^2$

Note that since $\hat{\beta}$ is a vector, $(X^T X)^{-1}\sigma^2$ is a variance-covariance matrix. Sometimes you want the standard error for a particular component which can be picked out as in $se(\hat{\beta}_i) = \sqrt{(X^T X)^{-1}_{ii}}\hat{\sigma}$.

## 2.10 Estimating $\sigma^2$

Recall that the residual sum of squares was $\hat{\varepsilon}^T\hat{\varepsilon} = y^T(I-H)y$. Now after some calculation, one can show that $E\hat{\varepsilon}^T\hat{\varepsilon} = \sigma^2(n-p)$ which shows that

$$\hat{\sigma}^2 = \frac{\hat{\varepsilon}^T\hat{\varepsilon}}{n-p}$$

is an unbiased estimate of $\sigma^2$. $n-p$ is the *degrees of freedom* of the model. Actually a theorem parallel to the Gauss-Markov theorem shows that it has the minimum variance among all quadratic unbiased estimators of $\sigma^2$.

## 2.11 Goodness of Fit

How well does the model fit the data? One measure is $R^2$, the so-called *coefficient of determination* or *percentage of variance explained*

$$R^2 = 1 - \frac{\sum(\hat{y}_i - y_i)^2}{\sum(y_i - \bar{y})^2} = 1 - \frac{\text{RSS}}{\text{Total SS(corrected for mean)}}$$

An alternative measure of fit is $\hat{\sigma}$. This quantity is directly related to the standard errors of estimates of $\beta$ and predictions. The advantage is that $\hat{\sigma}$ is measured in the units of the response and so may be directly interpreted in the context of the particular dataset. This may also be a disadvantage in that one must understand whether the practical significance of this measure whereas $R^2$, being unitless, is easy to understand.

## 2.12 Example

Now let's look at an example concerning the number of species of tortoise on the various Galapagos Islands. There are 30 cases (Islands) and 7 variables in the dataset. We start by reading the data into R and examining it

```
> data(gala)
> gala
             Species Endemics    Area Elevation Nearest Scruz Adjacent
      Baltra      58       23   25.09       346     0.6   0.6     1.84
   Bartolome      31       21    1.24       109     0.6  26.3   572.33

--- cases deleted ---

     Tortuga      16        8    1.24       186     6.8  50.9    17.95
        Wolf      21       12    2.85       253    34.1 254.7     2.33
```

The variables are

**`Species`** The number of species of tortoise found on the island

**`Endemics`** The number of endemic species

**`Elevation`** The highest elevation of the island (m)

**`Nearest`** The distance from the nearest island (km)

**`Scruz`** The distance from Santa Cruz island (km)

**`Adjacent`** The area of the adjacent island ($km^2$)

The data were presented by Johnson and Raven (1973) and also appear in Weisberg (1985). I have filled in some missing values for simplicity (see Chapter 14 for how this can be done). Fitting a linear model in R is done using the `lm()` command. Notice the syntax for specifying the predictors in the model. This is the so-called *Wilkinson-Rogers* notation. In this case, since all the variables are in the gala data frame, we must use the `data=` argument:

```
> gfit <- lm(Species ~ Area + Elevation + Nearest + Scruz + Adjacent,
     data=gala)
> summary(gfit)
Call:
lm(formula = Species ~ Area + Elevation + Nearest + Scruz + Adjacent,
    data = gala)

Residuals:
    Min      1Q  Median      3Q     Max
-111.68  -34.90   -7.86   33.46  182.58
```

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  7.06822   19.15420    0.37   0.7154
Area        -0.02394    0.02242   -1.07   0.2963
Elevation    0.31946    0.05366    5.95  3.8e-06
Nearest      0.00914    1.05414    0.01   0.9932
Scruz       -0.24052    0.21540   -1.12   0.2752
Adjacent    -0.07480    0.01770   -4.23   0.0003

Residual standard error: 61 on 24 degrees of freedom
Multiple R-Squared: 0.766,      Adjusted R-squared: 0.717
F-statistic: 15.7 on 5 and 24 degrees of freedom,       p-value: 6.84e-07
```

We can identify several useful quantities in this output. Other statistical packages tend to produce output quite similar to this. One useful feature of R is that it is possible to directly calculate quantities of interest. Of course, it is not necessary here because the `lm()` function does the job but it is very useful when the statistic you want is not part of the pre-packaged functions.

First we make the X-matrix

```
> x <- cbind(1,gala[,-c(1,2)])
```

and here's the response y:

```
> y <- gala$Species
```

Now let's construct $X^TX$: `t()` does transpose and `%*%` does matrix multiplication:

```
> t(x) %*% x
Error: %*% requires numeric matrix/vector arguments
```

Gives a somewhat cryptic error. The problem is that matrix arithmetic can only be done with numeric values but x here derives from the data frame type. Data frames are allowed to contain character variables which would disallow matrix arithmetic. We need to force x into the matrix form:

```
> x <- as.matrix(x)
> t(x) %*% x
```

Inverses can be taken using the `solve()` command:

```
> xtxi <- solve(t(x) %*% x)
> xtxi
```

A somewhat more direct way to get $(X^TX)^{-1}$ is as follows:

```
> gfit <- lm(Species ~ Area + Elevation + Nearest + Scruz + Adjacent,
                       data=gala)
> gs <- summary(gfit)
> gs$cov.unscaled
```

The `names()` command is the way to see the components of an Splus object - you can see that there are other useful quantities that are directly available:

```
> names(gs)
> names(gfit)
```

In particular, the fitted (or predicted) values and residuals are

```
> gfit$fit
> gfit$res
```

We can get $\hat{\beta}$ directly:

```
> xtxi %*% t(x) %*% y
          [,1]
[1,]  7.068221
[2,] -0.023938
[3,]  0.319465
[4,]  0.009144
[5,] -0.240524
[6,] -0.074805
```

or in a computationally efficient and stable manner:

```
> solve(t(x) %*% x, t(x) %*% y)
          [,1]
[1,]  7.068221
[2,] -0.023938
[3,]  0.319465
[4,]  0.009144
[5,] -0.240524
[6,] -0.074805
```

We can estimate $\sigma$ using the estimator in the text:

```
> sqrt(sum(gfit$res^2)/(30-6))
[1] 60.975
```

Compare this to the results above.

We may also obtain the standard errors for the coefficients. Also `diag()` returns the diagonal of a matrix):

```
> sqrt(diag(xtxi))*60.975
[1] 19.154139  0.022422  0.053663  1.054133  0.215402  0.017700
```

Finally we may compute $R^2$:

```
> 1-sum(gfit$res^2)/sum((y-mean(y))^2)
[1] 0.76585
```